

B5 Series B

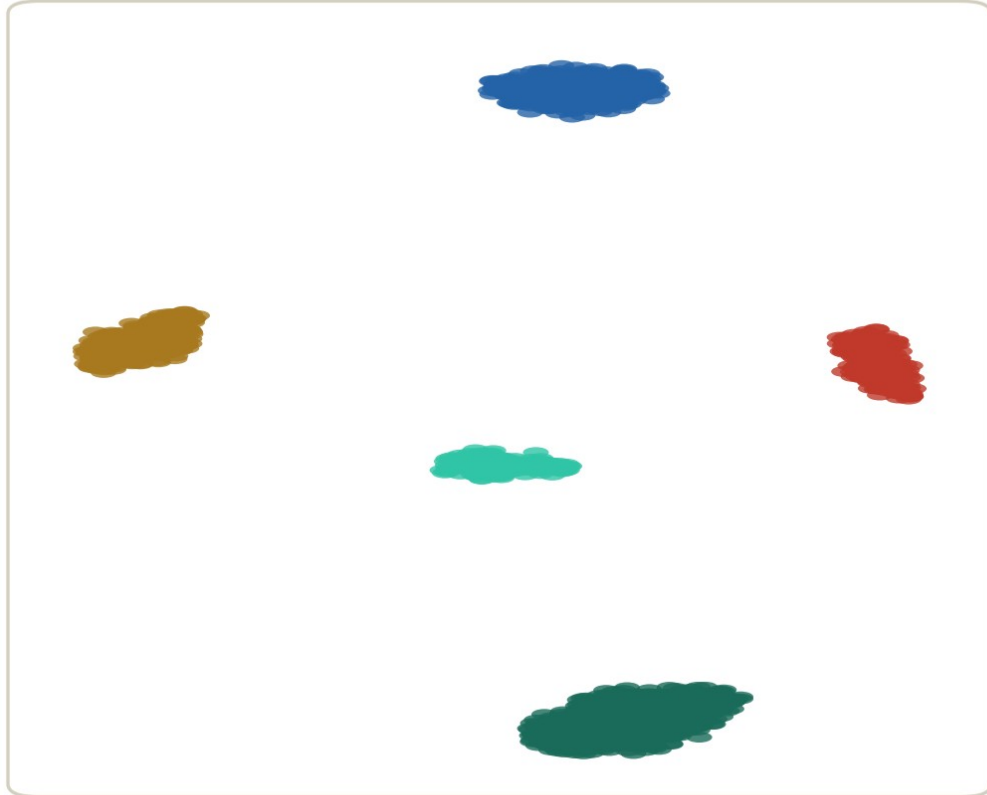
Loading data and QC: how thresholds are actually chosen

Set cutoffs from your data's distributions — not from someone else's paper.

About 40 min | Prerequisites: B1 | Data: PBMC 3k

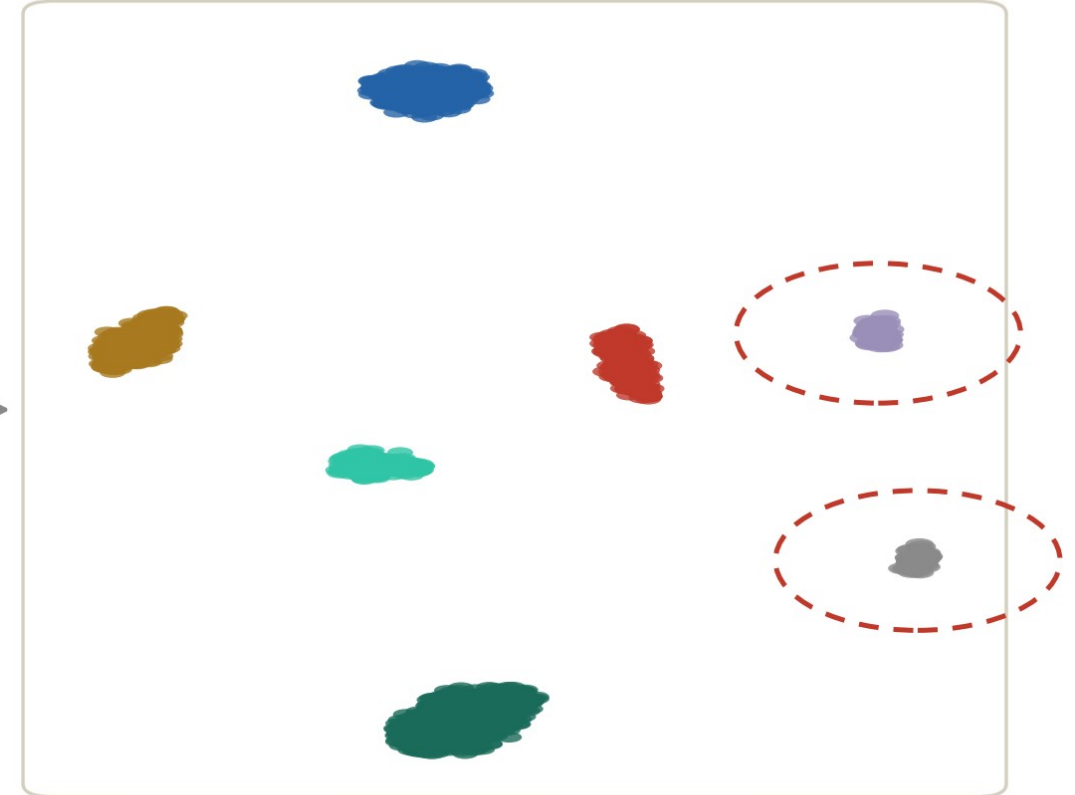
Same data, two QC thresholds

strict QC → 5 clusters



simulated data

loose QC → 7 clusters



the extra "clusters" are unfiltered junk

Left: strict, 5 clusters. Right: loose, 7 clusters. What are the extra two?

WHERE THIS EPISODE STARTS

Every QC cutoff decides what your downstream analysis can see.

Too loose and junk becomes a discovery; too harsh and rare populations vanish. Both are disasters.

What you will learn

- 1 Build a Seurat object from 10x output and compute the three QC metrics
- 2 Set thresholds from distributions — violins, scatters, MAD — with reasons you can defend
- 3 Run DoubletFinder and fold the result into your filtering decisions

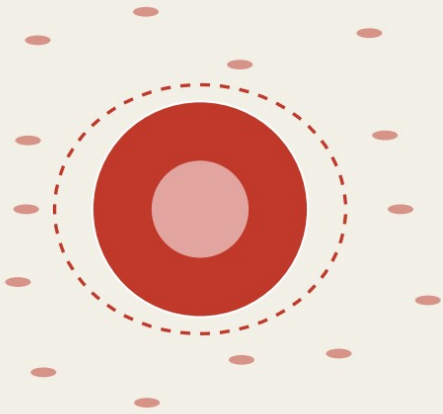
01

What QC is hunting

Three kinds of junk, three metrics — map them first

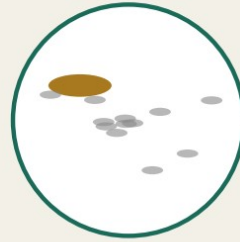
Three kinds of junk in your matrix

dying / broken cell



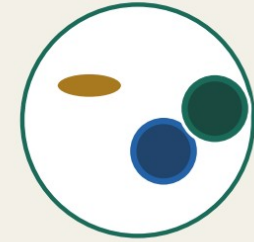
membrane leaks cytoplasmic RNA
mitochondrial fraction spikes

empty droplet + ambient RNA



no cell, only background RNA
very low counts and genes

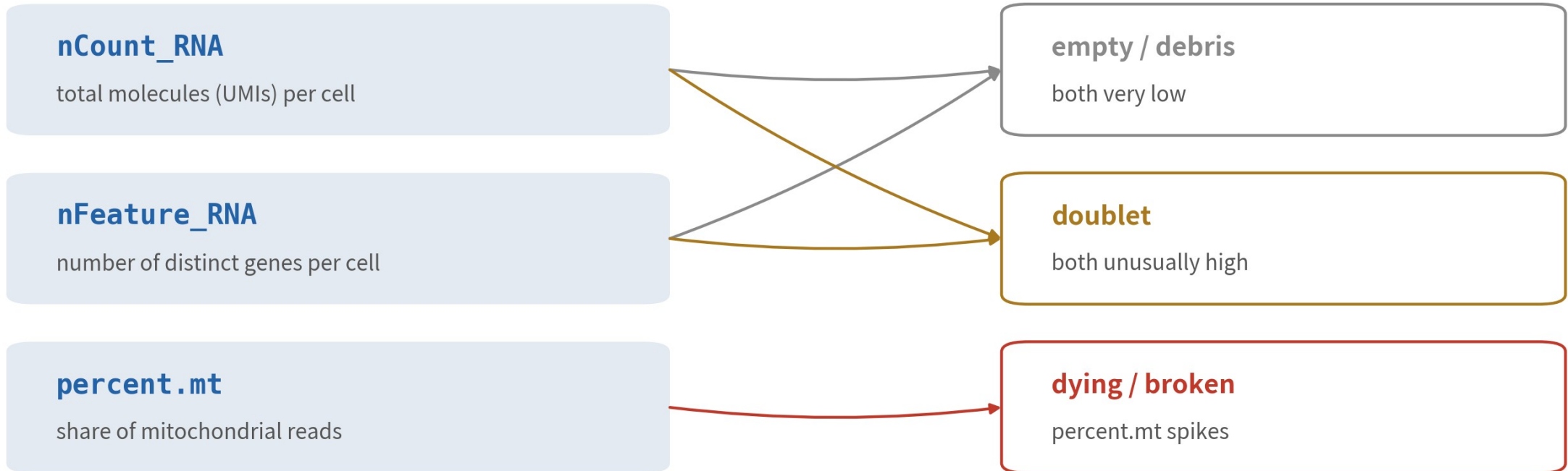
doublet



two cells in one droplet
counts and genes both high

Each gets a barcode and sits in the matrix pretending to be a cell

Three metrics, each with a target



three metrics, three kinds of bad cells

- nCount_RNA: total molecules (UMIs) per cell
- nFeature_RNA: distinct genes per cell
- percent.mt: mitochondrial share — it spikes as cells die
- both low → empty; both high → doublet; high mt → dying

CORE IDEA

**A threshold is a property of your dataset,
not a constant you copy around.**

The same 5% line is sensible for PBMC and catastrophic for cardiac tissue. You will see it happen.

02

Build the object, compute the metrics

Read10X → CreateSeuratObject → PercentageFeatureSet

From 10x output to a Seurat object

```
library(Seurat)
set.seed(1234) # 全系列固定，結果才能重現

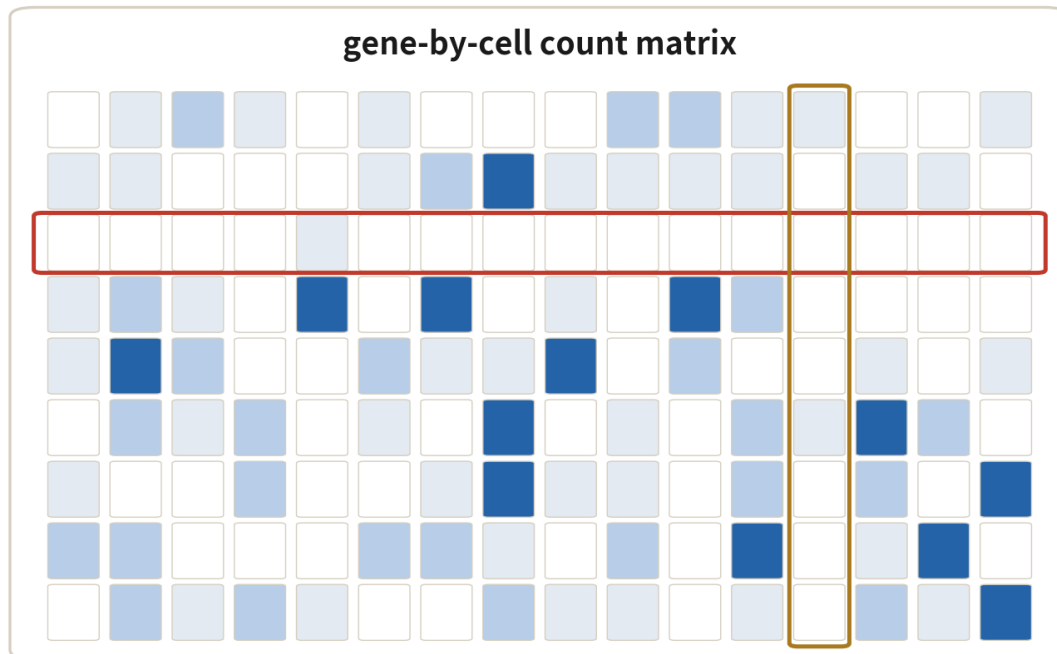
pbmc.data <- Read10X(data.dir =
  "data/pbmc3k/filtered_gene_bc_matrices/hg19/")
pbmc <- CreateSeuratObject(counts = pbmc.data, project = "pbmc3k",
  min.cells = 3, min.features = 200)

pbmc
```

An object of class Seurat
13714 features across 2700 samples within 1 assay
Active assay: RNA (13714 features, 0 variable features)

Read10X takes the folder holding the three files, not a single file path

What min.cells and min.features do



min.cells = 3

a gene seen in only 1-2 cells → drop the row
(almost surely noise; it slows every later step)

min.features = 200

a cell with too few genes → drop the column
(mostly empty droplets; a first coarse filter)

this is a coarse screen at object creation;
real QC thresholds come from the distributions.

- min.cells = 3: drop genes seen in fewer than 3 cells
- min.features = 200: drop cells with fewer than 200 genes
- this is a coarse screen — the real thresholds come from distributions

Compute percent.mt

```
pbm c[["percent.mt"]] <- PercentageFeatureSet(pbm c,  
                                              pattern = "^MT-")  
head(pbm c@meta.data, 3)
```

orig.ident	nCount_RNA	nFeature_RNA	percent.mt
AAACATACAACCAC -1	pbm c3k	2419	779 3.0177759
AAACATTGAGCTAC -1	pbm c3k	4903	1352 3.7935958
AAACATTGATCAGC -1	pbm c3k	3147	1129 0.8897363

Human mito genes are MT-xxx; in mouse use pattern "^mt-"

What you have at this point

One matrix



13,714 genes $\times$ 2,700
cells in the RNA assay

One metadata table



one row per cell:
nCount, nFeature,
percent.mt

No decisions yet



metrics ready, nothing
filtered — look first

03

Read the distributions, set the cutoffs

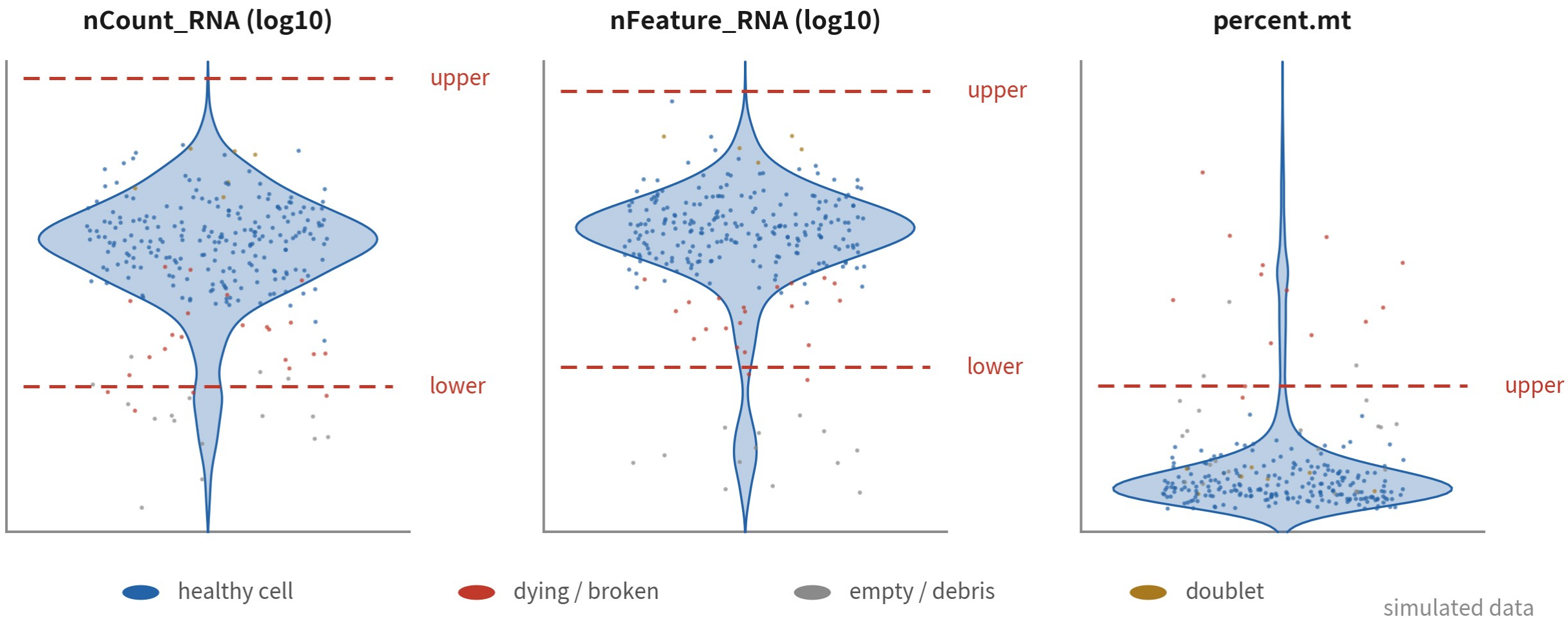
Look first, cut second — never the other way

The three distributions

```
ViolinPlot(pbm c,  
  features = c("nFeature_RNA", "nCount_RNA", "percent.mt"),  
  ncol= 3)
```

These three violins are the most important picture of the episode

How to read the violins



The body is healthy cells; the lower tail is debris, the upper outliers are doublets and dying cells

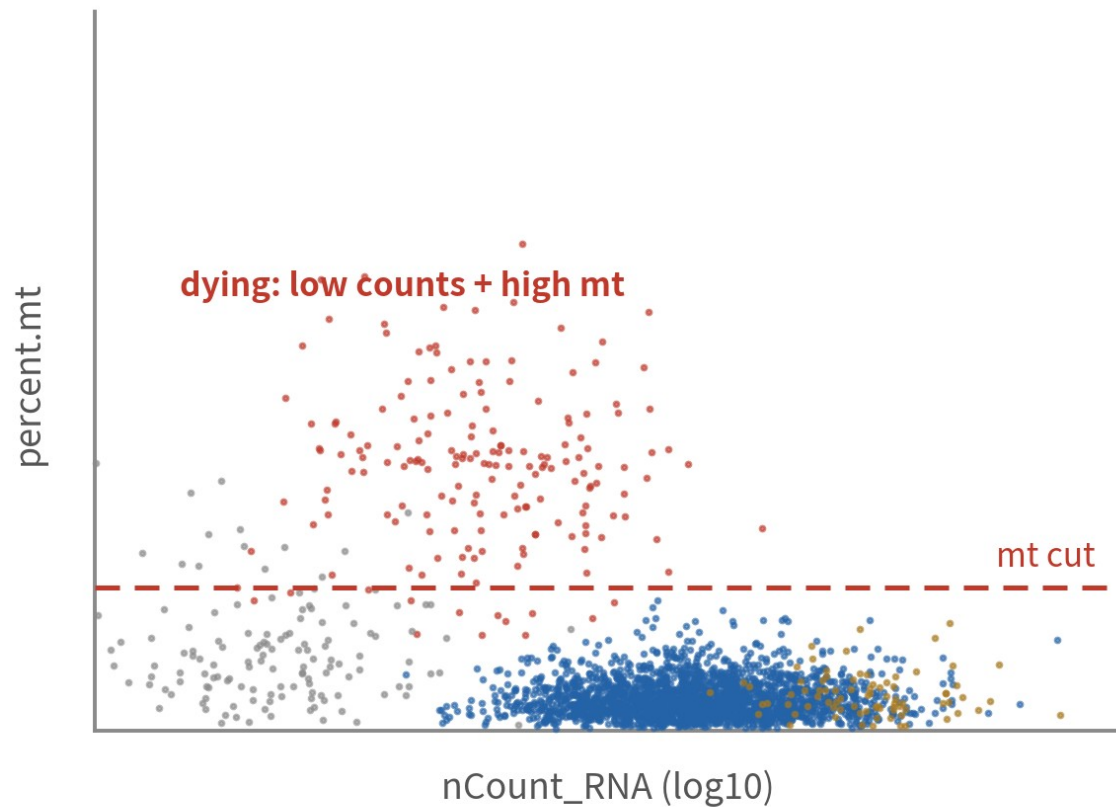
Two scatters, read together

```
FeatureScatter(pbm c, feature1 = "nCount_RNA",  
               feature2 = "percent.m t")  
FeatureScatter(pbm c, feature1 = "nCount_RNA",  
               feature2 = "nFeature_RNA")
```

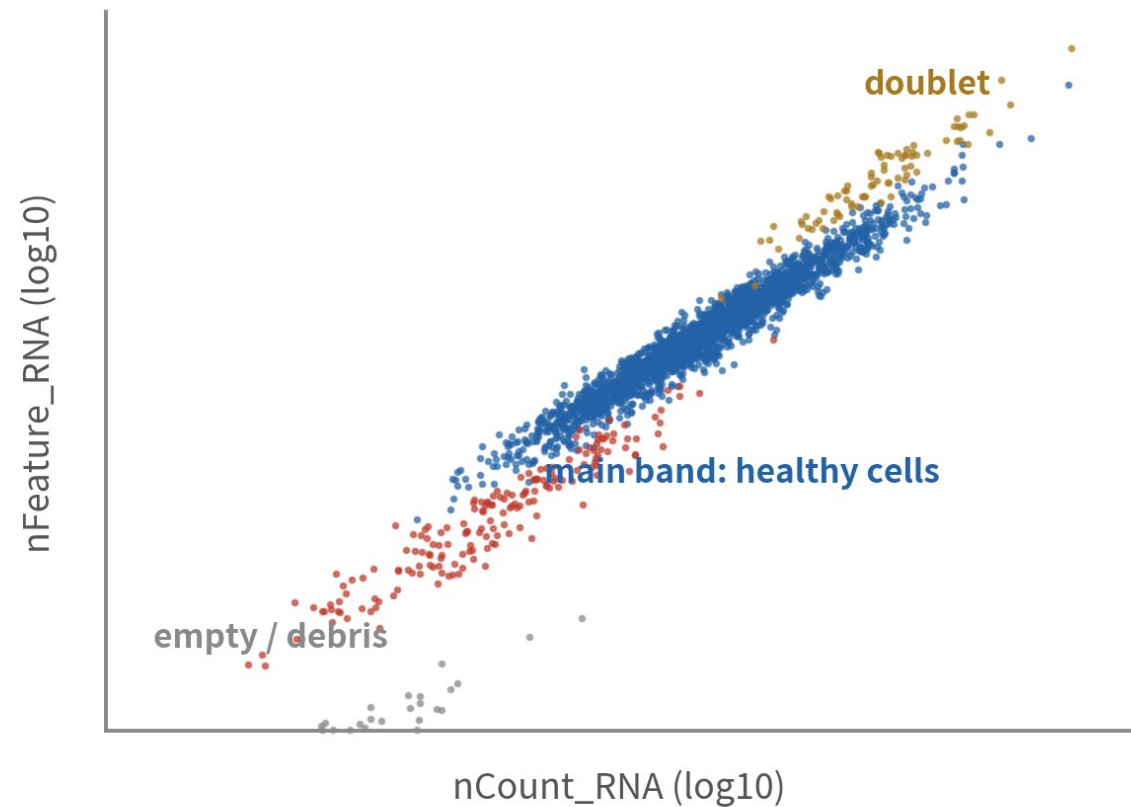
Single metrics mislead; in pairs, each kind of junk owns a corner

The corners of the scatter

nCount vs percent.mt



nCount vs nFeature



simulated data

Left: dying cells float high in mt. Right: main band on the diagonal, debris bottom-left, doublet candidates top-right

Two ways to set a cutoff

The point is not the camp you pick — it is whether you can defend it

fixed

Eyeball the plot, draw a line

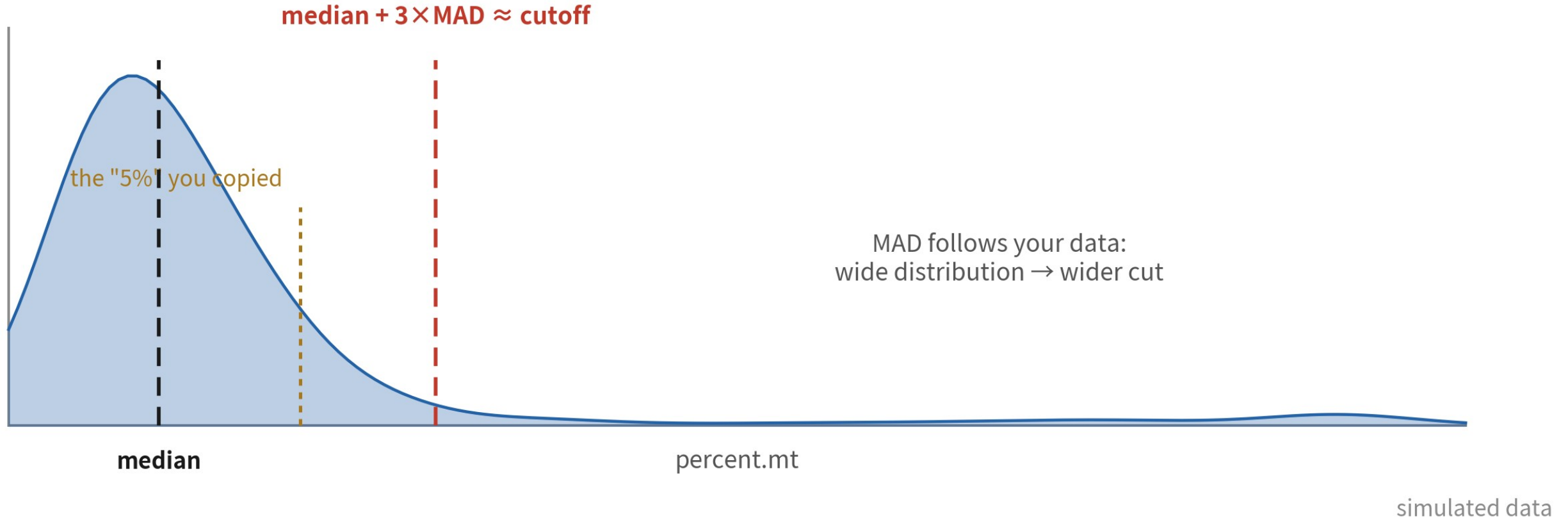
- e.g. nFeature 200–2500, mt < 5%
- intuitive, easy to communicate
- but must be re-derived per dataset

MAD

median $\pm 3 \times \text{MAD}$

- follows the distribution, robust to outliers
- scriptable — shines with many samples
- log first for skewed metrics

The MAD line follows the data



The same rule yields different lines on different data — exactly why it beats copying 5%

A MAD threshold in three lines

```
m t <- pbmc.c$percent.m t  
m t.upper <- median(m t) + 3 * mad(m t) # mad() 已含 1.4826 校正  
m t.upper
```

```
[1] 4.91
```

For PBMC 3k it lands near 5% — computed, not copied

FOR YOUR METHODS SECTION

**"Cells were retained with nFeature 200–2500
and percent.mt < median + 3×MAD (= 4.9%)."**

Every line has a number and a reason. When a reviewer asks why, you have an answer.

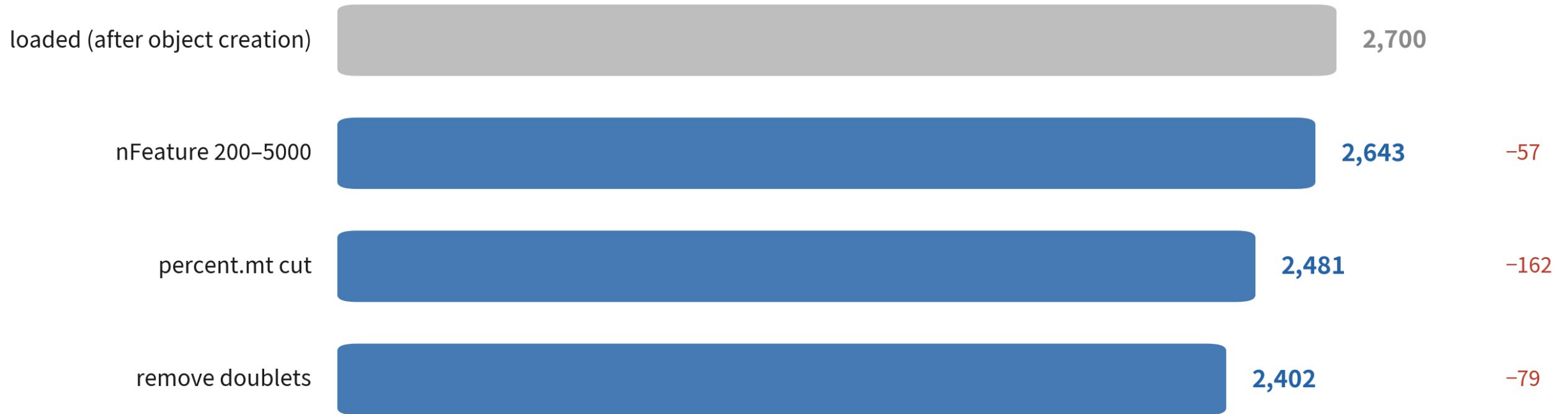
Apply the filter

```
pbm c <- subset(pbm c,  
  subset= nFeature_RNA > 200 &  
    nFeature_RNA < 2500 &  
    percent_m_t < m_t.upper)  
pbm c
```

An object of class Seurat
13714 features across 2638 samples within 1 assay

2700 → 2638: 62 cells removed. Write that number down

Record what every step removes



record what each step removes — this goes in your methods section

If any step removes a suspiciously large share (>20-30%), question the sample or the cutoff before moving on

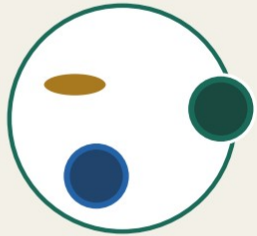
04

Doublets: the junk your metrics miss

High counts are only a suspicion — you need a dedicated tool

Two kinds of doublet, wildly different difficulty

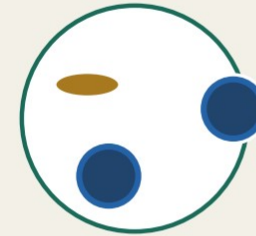
heterotypic doublet



two different cell types
profile mixes A and B

catchable

homotypic doublet



two cells of the same type
looks like one oversized cell

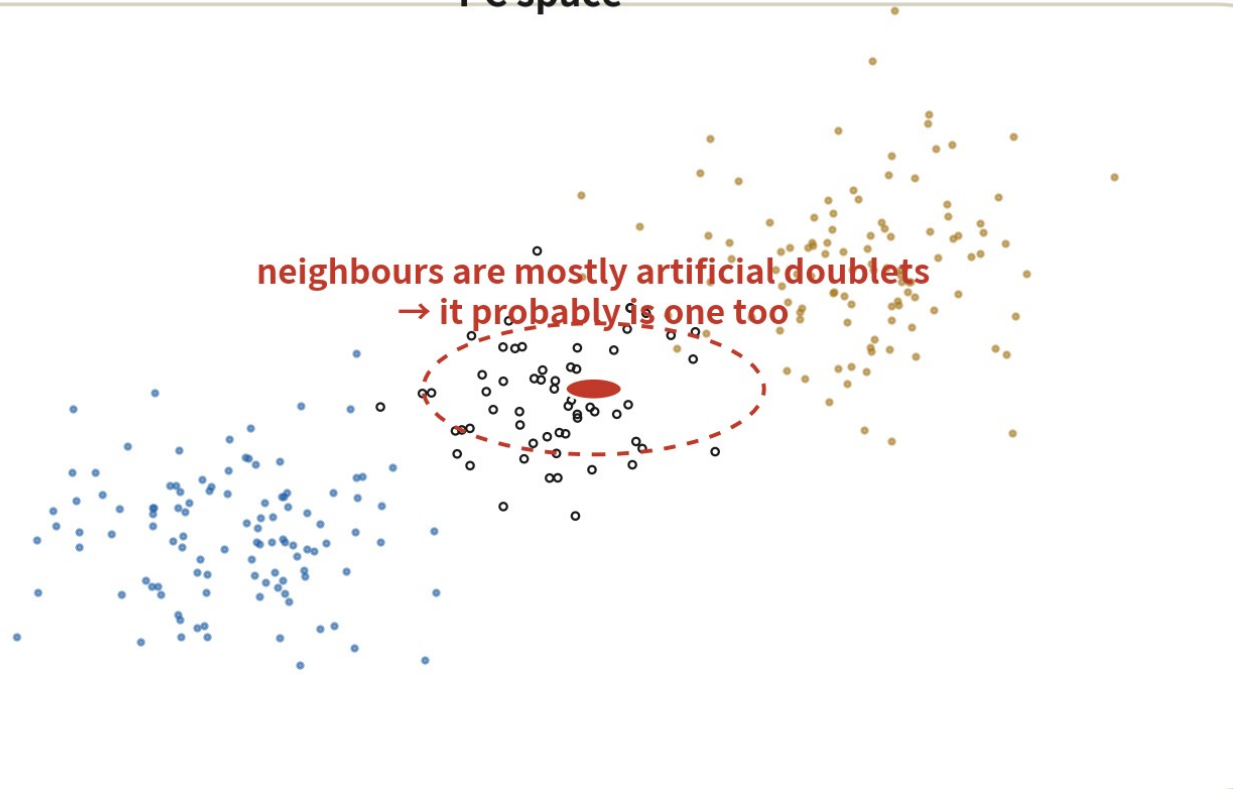
nearly invisible

DoubletFinder mainly catches heterotypic doublets; homotypic ones need hashing

Algorithms mainly catch heterotypic ones; homotypic doublets need experimental design (hashing)

The intuition behind DoubletFinder

PC space



1. make fakes

average random pairs of real cells
and inject them back

2. check neighbours

for each real cell, count the share of
artificial doublets among its neighbours (pANN)

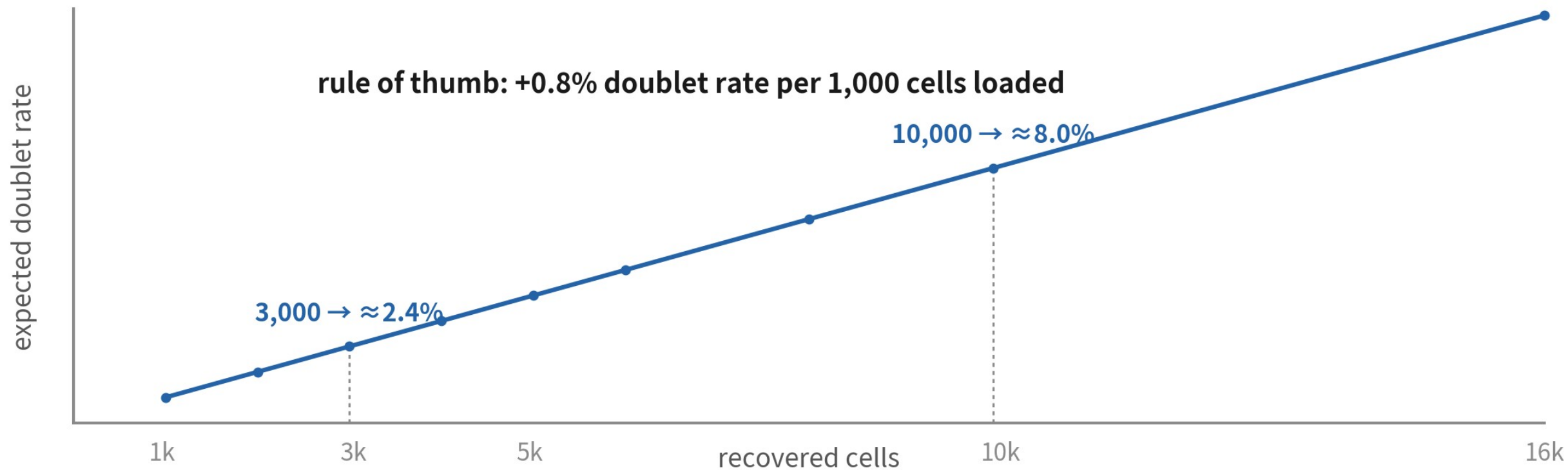
3. cut by expected rate

expected rate from loading tables
decides how many get flagged

simulated data

Manufacture fake doublets, then see who hangs out with them

How many to flag? Look it up



The expected rate comes from loading, not from the algorithm — you must supply it

DoubletFinder: prep and pK sweep

```
# remotes::install_github("chris-mcginnis-ucsf/DoubletFinder")
library(DoubletFinder)
# 需要先跑完標準前處理——每一步 B6-B9 都會講透，這裡先照抄
pbmc <- NormalizeData(pbmc) |> FindVariableFeatures() |>
  ScaleData() |> RunPCA() |> RunUMAP(dims = 1:10)

sweep.res <- paramSweep(pbmc, PCs = 1:10, sct = FALSE)
sweep.stat <- summarizeSweep(sweep.res, GT = FALSE)
bcm.vn <- find.pK(sweep.stat) # 挑 BC metric 最高的 pK
```

pK sets the neighbourhood size; choose it by sweeping, not guessing

DoubletFinder: run and inspect

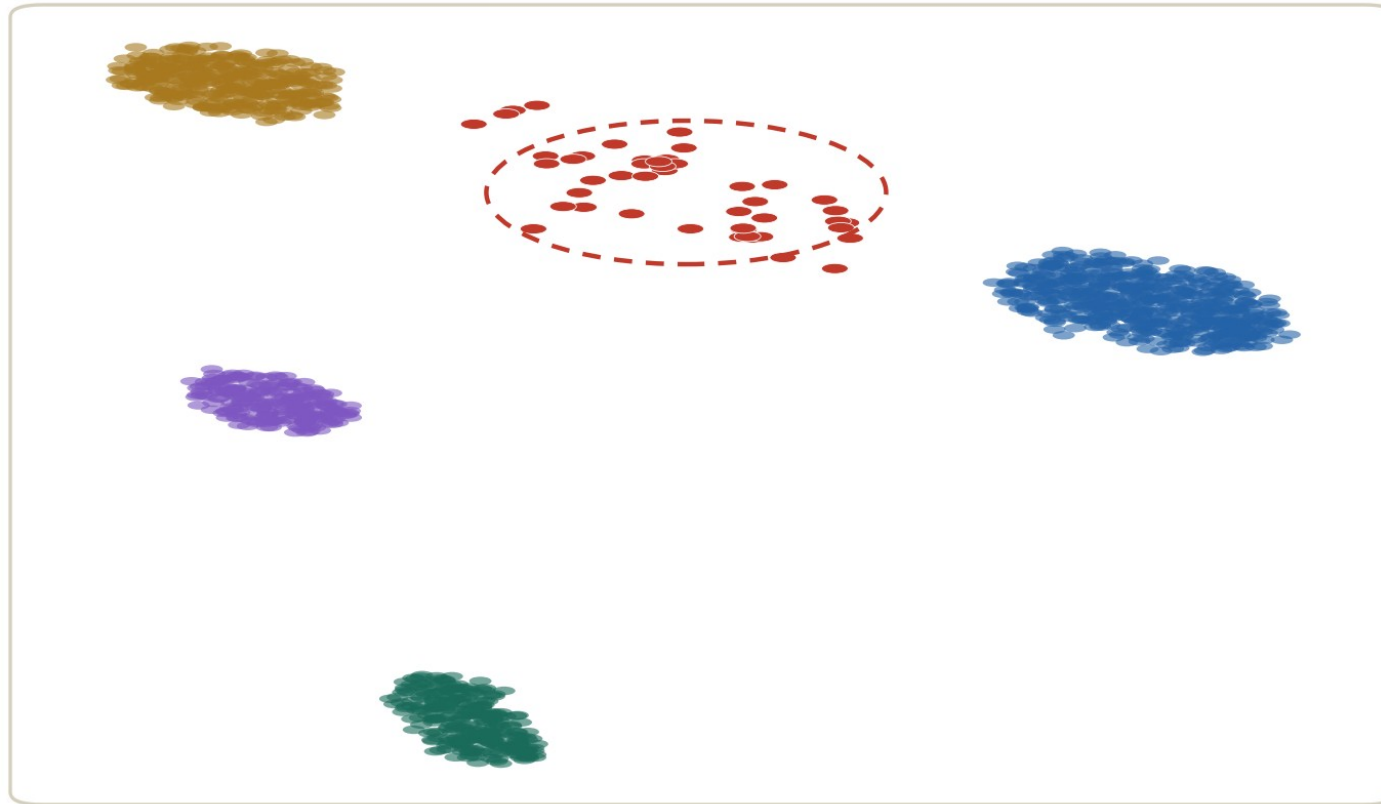
```
nExp <- round(0.023 * ncol(pbm c)) # ~2,600 顆 → 預期約 2.3%  
pbm c <- doubletFinder(pbm c, PCs = 1:10, pN = 0.25,  
                        pK = 0.09, nExp = nExp)  
table(pbm c$DF.classifications_0.25_0.09_61)
```

```
DoubletSinglet  
61    2577
```

Results land in a new metadata column named after the parameters

What doublets look like on the embedding

doublets (red) often bridge two clusters



schematic

Bridges between clusters — easily misread downstream as transitions or intermediate states

Once flagged: drop or keep?

Both are common practice — what matters is stating what you did

remove

subset out the doublets

- clean downstream, no lingering worry
- what most papers do
- a wrong expected rate kills good cells

flag

keep them, but flagged

- after clustering: an all-doublet cluster is suspect
- good for a first exploratory pass
- still remove before the final analysis

05

Where people get hurt

Three traps — each demonstrated for real

Trap 1: a flat 5% mito cut

What it is

Copying the tutorial's 5% with no look at tissue or distribution.



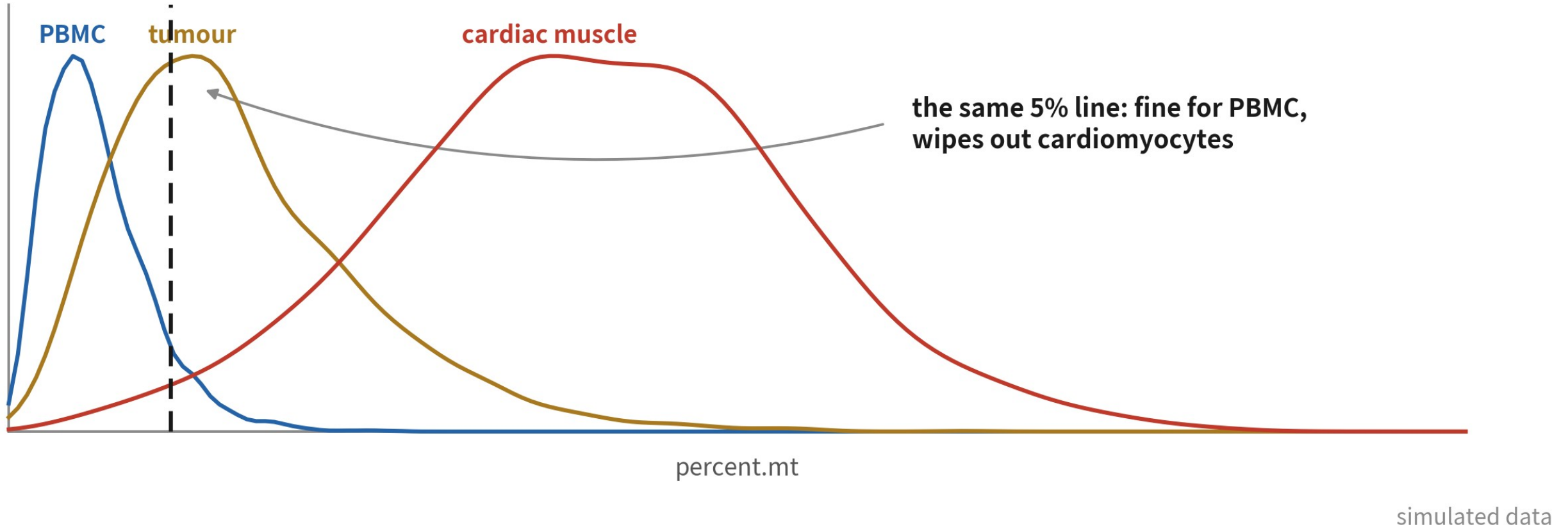
Why it happens

Tutorials use PBMC, where 5% happens to work; your tissue may not be PBMC.

What it costs you

In high-metabolism tissue the entire cell type vanishes before analysis even starts.

One 5% line, three tissues



Normal mito fraction is tissue-specific. Look at the distribution before drawing the line

Trap 2: filter first, look later

What it is

Apply copied thresholds immediately, and only plot distributions afterwards.



Why it happens

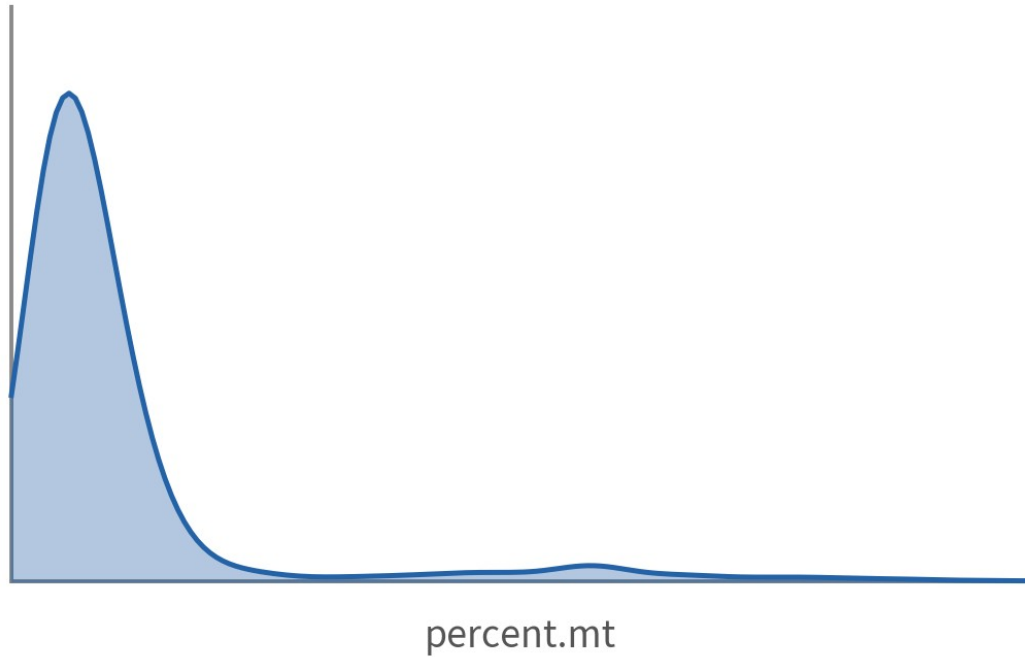
The order feels harmless — you plot the data either way.

What it costs you

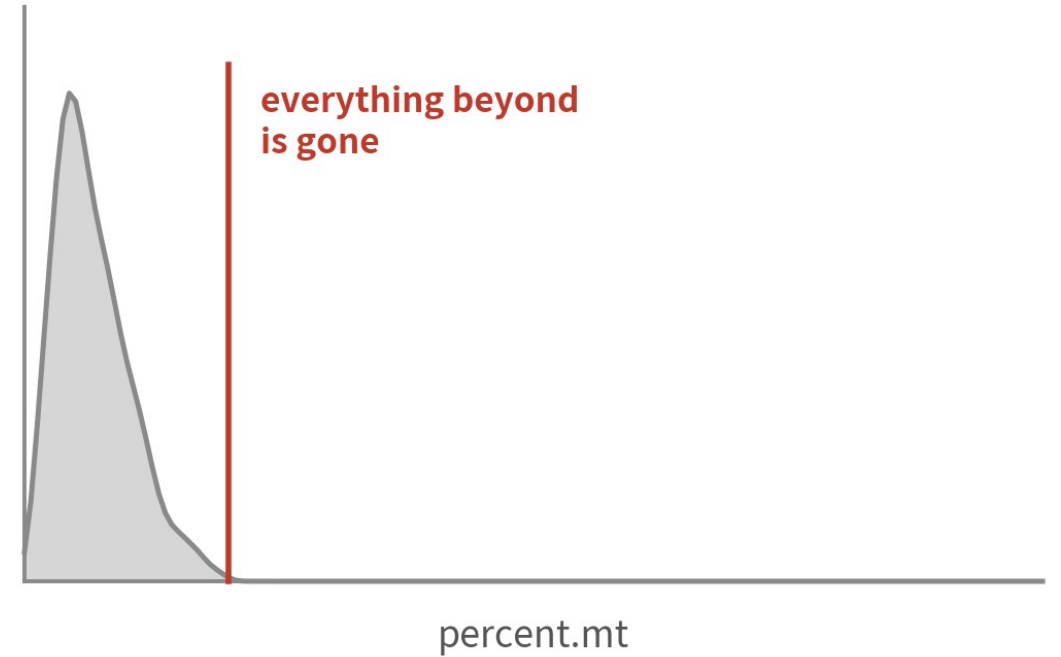
The distribution is truncated and the tail is gone — you can no longer tell where the line should have been, or that it is wrong.

After truncation there is no going back

look first: you can see the tail



filter first: the tail is gone



simulated data

Save the unfiltered object first: `saveRDS(pbm, "output/pbm_raw.rds")`

Trap 3: QC as a one-shot gate

What it is

Filter once, never look back, run straight to DE.



Why it happens

Pipelines draw QC as step one, so it feels like a gate you pass once.

What it costs you

Junk clusters that only surface after clustering — high mt, low nFeature, stress markers — get published as novel cell types.

QC is a loop, not a gate



QC is a loop, not a gate: filter → inspect → revisit

After clustering, revisit per-cluster QC — B8 does this check for real

TRAPS, SUMMED UP

**Every threshold is a hypothesis.
Write it down, and be ready to revise it.**

If you can state the cutoff, the reason, and the count removed, your QC passes.

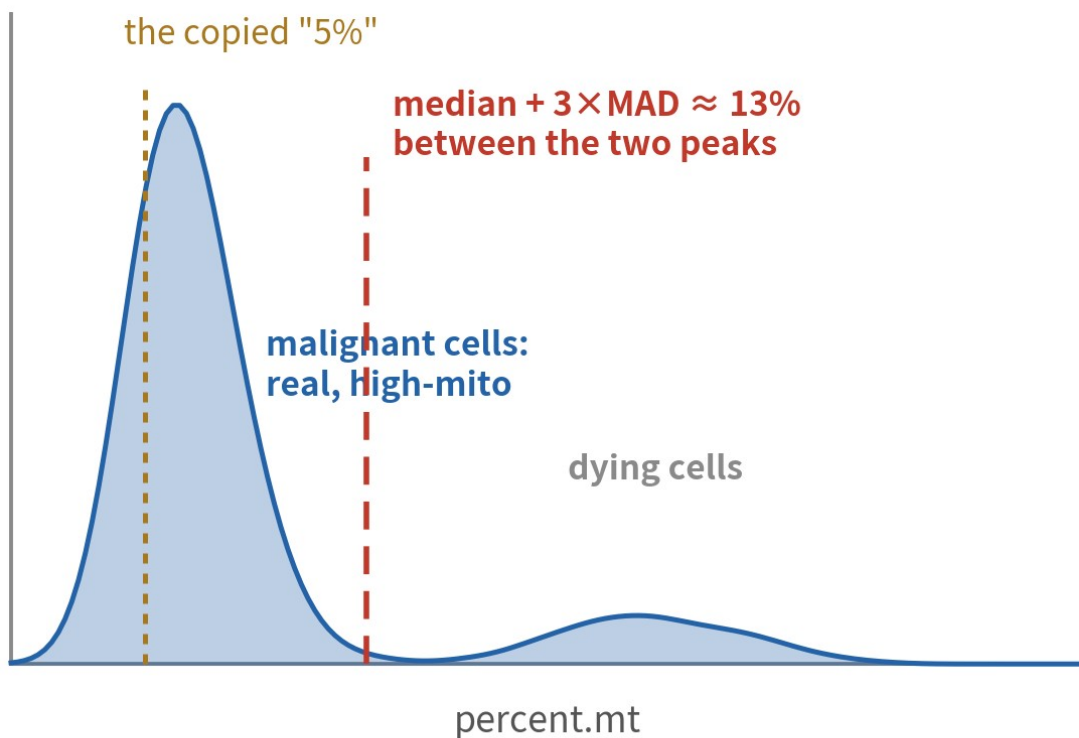
05+

Strategy Room: Complex Samples

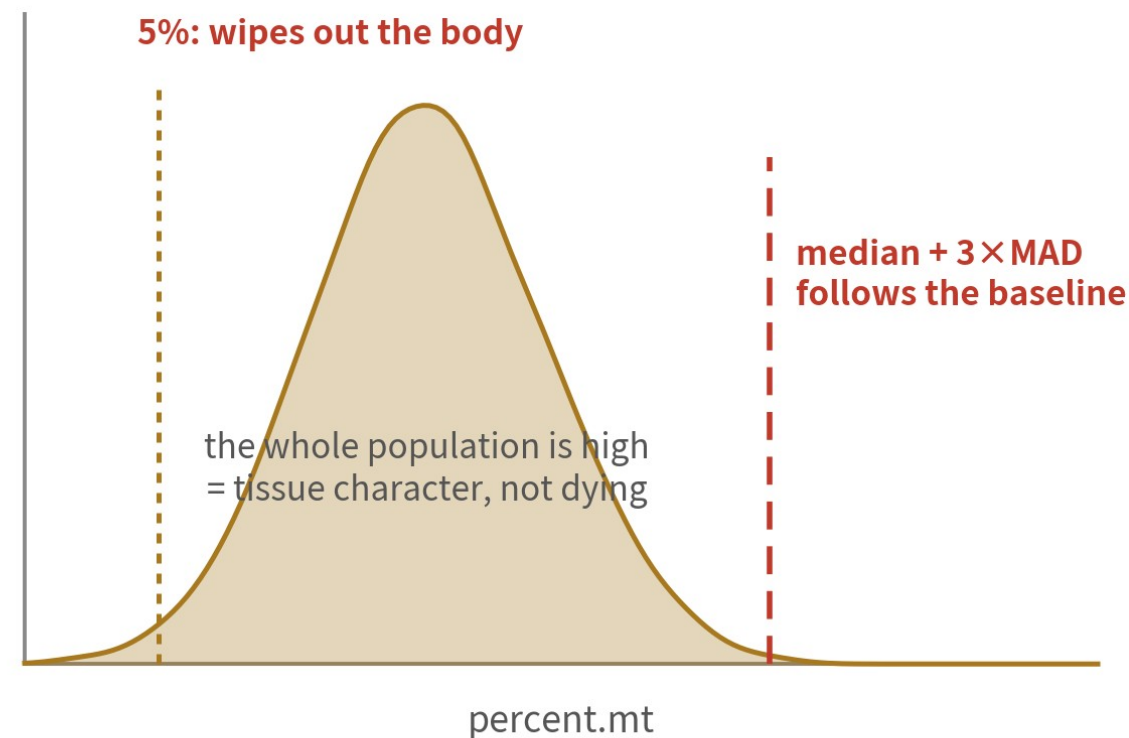
The same QC step on a tumour — which decisions change, and why

What tumour QC distributions look like

GBM: high body + second peak



liver cancer: baseline shifted up

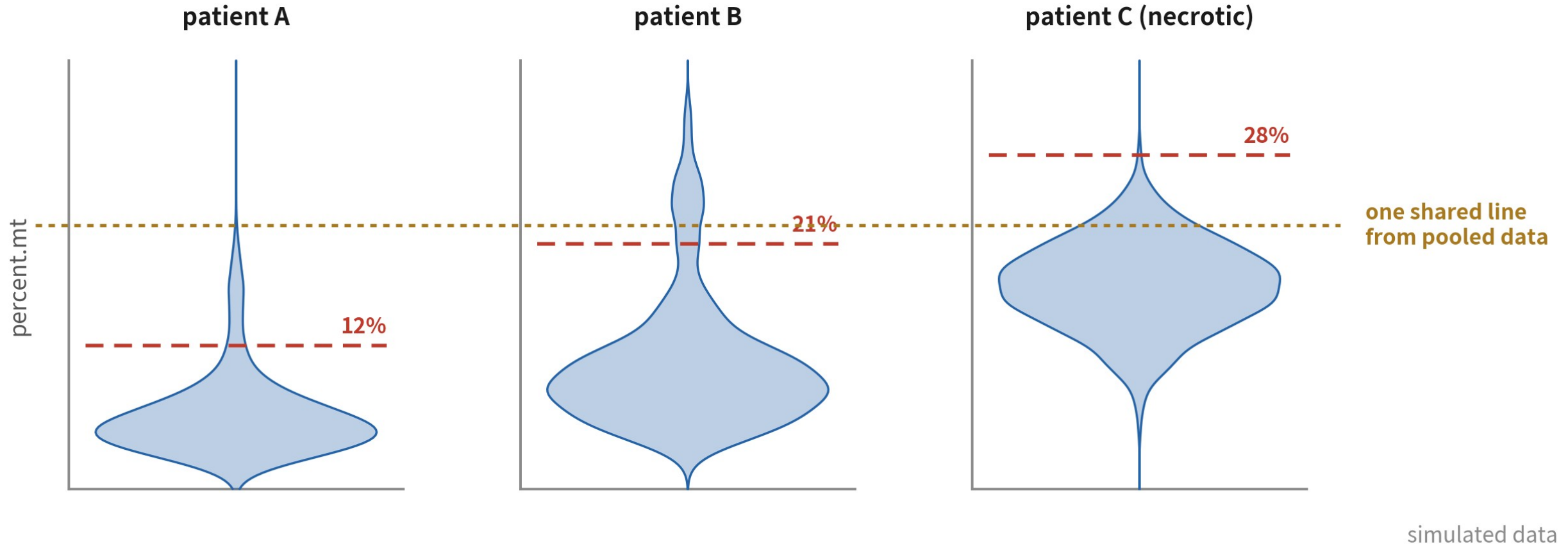


one MAD rule fits both tumours — the copied 5% fails both

simulated data

High-mito malignant cells are real cells: GBM's line sits between two peaks, liver cancer shifts the whole baseline — MAD follows both

Per-sample QC: one rule, one line per sample



shared gold line: too loose for A, cuts into C's body. Red = per-sample median + $3 \times \text{MAD}$

Standard for multi-sample tumour studies: share the rule, not the number — compute and record per sample

The dissociation-stress signature: FOS / JUN / HSP

Solid-tumour dissociation is long and rough — cells start emergency transcription in the tube

Spot it



Score the immediate-early set — FOS, JUN, heat-shock genes — and paint it on the UMAP

Read it



A light scatter everywhere is normal; a whole cluster defined by it is a stress cluster, not a new cell state

Handle it



Exclude the gene set when interpreting (B13 returns to this), or switch to cold dissociation upstream

SoupX: tumours brew a strong ambient-RNA soup

Estimate the soup from empty droplets, get each cell's contamination fraction ρ , then decide

$\rho < 5\%$

Safe to skip

- typical for clean samples — tutorial PBMC data lives here
- correction barely moves the conclusions
- but compute and record ρ — don't assume it

$\rho > 10\%$

Correct it

- common in necrotic, long-dissociation tumours
- uncorrected, every cluster 'expresses a bit of' other lineages' markers
- needs both raw and filtered matrices — keep your raw output

snRNA-seq: every QC threshold changes

Frozen tumours often run as nuclei — new material, inverted QC logic

Metric	scRNA-seq (whole cell)	snRNA-seq (nuclei)
percent.mt	baseline is tissue-specific; MAD sets the bound	near zero; high means cytoplasmic contamination — cut very low
Intronic signal	low fraction	high by design, not a warning; count introns (B3)
nCount / nFeature	whole-cell RNA	nuclear RNA only, lower overall — lower the floor too
Bottom line	tutorial numbers fit only their own data	carry over none of the scRNA cutoffs

Episode checklist



min.cells / min.features are a coarse screen only



plot all three metrics before choosing any threshold



every cutoff has a reason — fixed or MAD — written down



you recorded how many cells each step removed



doublet detection ran; results live in metadata



after clustering, you looked back for QC-artifact clusters

All six boxes ticked before you move to B6

ONE-LINE SUMMARY

Look at the distributions, then set the cutoffs

—

and write down why.

QC has no standard answer, but it has a standard procedure.

Check yourself 1

A spike in percent.mt mainly flags which kind of bad cell?

- A Empty droplets and debris
- B Dying cells with broken membranes
- C Doublets
- D Red blood cell contamination

Answer B. As a cell dies the membrane leaks cytoplasmic RNA while mitochondrial RNA remains, so the fraction spikes. Empty droplets show both metrics low; doublets show both high.

Check yourself 2

A small group with unusually high nCount and nFeature is most likely...

- A Healthy cells sequenced more deeply
- B Empty droplets
- C Doublet candidates
- D Mitochondrial contamination

Answer C. Two cells' RNA stacked together lifts both totals and gene counts. Deeper sequencing raises nCount, but nFeature saturates rather than jumping off the band.

Check yourself 3

Your cardiac dataset has a median percent.mt of 18%. How do you set the cutoff?

- A Use the conventional 5%
- B Use median + 3×MAD or a tissue-specific bound, and record the reason
- C Skip mito filtering entirely
- D Cut at 5% first, then look

Answer B. Cardiomyocytes are naturally mitochondria-rich; a 5% cut removes most healthy cells. Follow this dataset's distribution (e.g. median + 3×MAD) and record the number and reason.

NEXT EPISODE

B6: Normalization, HVG, Scaling

Your cells are clean, but totals differ ten-fold — comparing them raw means comparing depth.
Two routes: LogNormalize and SCTransform. We run both and compare downstream.